

Módulo 3 — Codificación Numérica

Ejercicios prácticos

Esta colección contiene 6 ejercicios que cubren los temas centrales del módulo: conversión Float $\leftrightarrow$ Fixed, suma y multiplicación en punto fijo, recortado con truncado/redondeo/saturación, recodificación CSD, multiplicación Booth radix-2, y aritmética en RNS.

Los ejercicios 2 a 6 cuentan con un módulo Verilog y un testbench para Icarus + GTKWave; los archivos están en `ejercicios/verilog/`.

#	Tema	Tipo
1	Conversión Float $\leftrightarrow$ Fixed	Cálculo
2	Suma en punto fijo signado	Cálculo + Verilog
3	Truncado, Redondeo y Saturación	Cálculo + Verilog
4	CSD — recodificación	Cálculo + Verilog
5	Booth radix-2	Cálculo + Verilog
6	RNS — multiplicación modular	Cálculo + Verilog

Ejercicio 1 Conversión Float $\leftrightarrow$ Fixed

Enunciado

Dado el número $x = +9,625$ en decimal:

- (a) Representar x en IEEE 754 simple precisión (32 bits): signo, exponente y mantisa.
- (b) Representar x en punto fijo S(8, 3).
- (c) Calcular el error de cuantización al pasar de float a fixed.

Datos

- $x = 9,625$
- Float: IEEE 754 simple (1+8+23 bits)
- Fixed: S(8, 3)

Entregables

- Pasos de cada conversión
- Bits explícitos en ambos formatos
- Error de cuantización en LSB y en valor absoluto

■ **Pista:** Normalizar primero el binario; el exponente lleva sesgo 127 (simple precisión).

Ejercicio 2 Suma en punto fijo

Enunciado

Dadas dos señales en formato signado (complemento a 2):

A = S(6, 4), bits: 110010

B = S(8, 5), bits: 00011110 (valor: +0,9375)

- (a) Determinar el formato del resultado $A + B$.
- (b) Calcular la suma binaria alineada y verificar el resultado decimal.

Datos

- A: $S(6, 4) = 110010$
- B: $S(8, 5) = 00011110$
- Reglas: $NBF_{out} = \max(NBF_1, NBF_2)$,
 $NBI_{out} = \max(NBI_1, NBI_2) + 1$

Entregables

- Bits de A y B alineados a la coma
- Suma en complemento a 2
- Verificación del valor decimal

■ **Pista:** Antes de sumar, extender signo y rellenar con ceros a la derecha para alinear NBF.

Ejercicio 3 Truncado, Redondeo y Saturación

Enunciado

(a) Se tiene $x = 5,5625$ en formato $S(10, 6)$. Recortarlo a $S(7, 3)$ por **truncado** y por **redondeo**. Calcular el error en cada caso.

(b) Se tiene $y = 8,75$ en formato $S(10, 6)$. Recortarlo a $S(5, 3)$ usando **wrap-around** y **saturación**. Comparar los resultados.

Datos

- $x = 5,5625$ en $S(10, 6) \rightarrow S(7, 3)$
- $y = 8,75$ en $S(10, 6) \rightarrow S(5, 3)$

Entregables

- Bits y valores recortados
- Errores: trunc vs round
- Resultado: wrap vs saturación

■ **Pista:** Rango de $S(5,3)$: $[-2, +1,875]$. Si $y > 1,875 \rightarrow \text{overflow}$.

Ejercicio 4 CSD — recodificación

Enunciado

Implementar la multiplicación por la constante $K = 23$ en hardware:

- (a) Expresar $K = 23$ en binario estándar.
- (b) Recodificarla en CSD canónico (dígitos $\{-1, 0, +1\}$, sin no-ceros consecutivos).
- (c) Escribir $Y = X \cdot 23$ como sumas/restas y shifts. Comparar la cantidad de sumadores en cada forma.

Datos

- $K = 23$
- Objetivo: minimizar cantidad de sumadores en $Y = X \cdot K$

Entregables

- K en binario
- K en CSD
- Expresión $Y = X \cdot K$ con shifts y $\pm$
- Comparación de sumadores

■ **Pista:** Regla típica: reemplazar "0111...1" por "100...0(-1)" (un "1" más alto + un "-1" al final).

Ejercicio 5 Booth radix-2

Enunciado

Multiplicar $A \cdot B$ usando el algoritmo de Booth radix-2 con:

$A = +6$ (4 bits, C2)

$B = -5$ (4 bits, C2)

- (a) Expresar A y B en complemento a 2 (4 bits).
- (b) Aplicar Booth radix-2: tabla de pares (b_i, b_{i-1}) , acción y producto parcial.
- (c) Verificar que el resultado coincide con $A \cdot B = -30$.

Datos

- $A = +6$ (4 bits, C2)
- $B = -5$ (4 bits, C2)
- Agregar bit $b_{-1} = 0$

Entregables

- Bits A, B en C2
- Tabla de pasos Booth
- Suma de productos parciales
- Verificación $A \cdot B = -30$

■ **Pista:** Pares (b_i, b_{i-1}) : $01 \rightarrow +A \ll i$; $10 \rightarrow -A \ll i$; 00 o $11 \rightarrow 0$.

Ejercicio 6 RNS — multiplicación modular

Enunciado

Trabajando en RNS con módulos $\{ 3, 5, 7 \}$ ($M = 105$):

- (a) Representar $X = 14$ e $Y = 6$ en RNS.
- (b) Calcular $X \cdot Y$ residuo a residuo, en paralelo.
- (c) Reconponer el resultado decimal y verificar que coincida con $X \cdot Y = 84$.

Datos

- Módulos: $\{3, 5, 7\}$
- $M = 3 \cdot 5 \cdot 7 = 105$
- $X = 14, Y = 6$

Entregables

- X e Y en RNS
- Producto modular residuo a residuo
- Reconposición decimal

■ **Pista:** $c_i = (a_i \cdot b_i) \bmod m_i$ sin propagación de carry entre canales.

Los ejercicios 2 a 6 de esta colección incluyen testbenches **self-checking**: en lugar de inspeccionar visualmente formas de onda, comparan la salida del DUT contra una **referencia "golden"** generada por una librería Python que modela aritmética en punto fijo.

La librería que usamos es **fxpmath**. Reproduce con exactitud el formato S(NB, NBF) que estudiamos en el módulo, con control fino de overflow (wrap / saturate) y de redondeo (trunc / around / etc).

Instalación

fxpmath se instala con pip:

```
pip3 install --user fxpmath
```

Compatible con Python ≥ 3.7 . No necesita compilación: es Python puro apoyado en NumPy.

Uso básico

La clase principal es Fxp. Recibe un valor y los parámetros del formato:

```
from fxpmath import Fxp

x = Fxp(9.625, signed=True, n_word=8, n_frac=3) # S(8, 3)
print(x.bin()) # '01001101'
print(x.hex()) # '4d'
print(x.get_val()) # 9.625
```

Parámetros del constructor

Parámetro	Valores	Significado
signed	True / False	Aritmética en C2 o sin signo
n_word	entero ≥ 1	NB (bits totales)
n_frac	entero	NBF (bits fraccionales; puede ser negativo)
overflow	"wrap" / "saturate"	Qué hacer si el valor sale del rango
rounding	"trunc" / "around"	Cómo eliminar bits LSB sobrantes

Ejemplos relevantes para los ejercicios

Suma en punto fijo (Ej. 2)

```
A = Fxp(-0.875, signed=True, n_word=6, n_frac=4)
B = Fxp(0.9375, signed=True, n_word=8, n_frac=5)
S = Fxp(A.get_val() + B.get_val(), signed=True, n_word=9, n_frac=5)
print(S.bin(), S.get_val()) # 000000010 0.0625
```

Trunc vs Round, Wrap vs Sat (Ej. 3)

```
x = 5.5625
tw = Fxp(x, signed=True, n_word=7, n_frac=3, overflow='wrap', rounding='trunc')
rs = Fxp(x, signed=True, n_word=7, n_frac=3, overflow='saturate',
rounding='around')
print(tw.get_val(), rs.get_val()) # 5.5 5.625
```

Multiplicación signada (Ej. 4 y 5)


```
A = Fxp(6, signed=True, n_word=4)
B = Fxp(-5, signed=True, n_word=4)
P = Fxp(A.get_val() * B.get_val(), signed=True, n_word=8)
print(P.get_val()) # -30
```

Integración con los testbenches Verilog

En cada carpeta `verilog/ejNN_xxx/` hay 4 archivos:

Archivo	Contenido
<code>gen_vectors.py</code>	Genera los vectores <code>.hex</code> usando <code>fxpmath</code> como referencia
<code>*.v (DUT)</code>	Módulo Verilog a verificar
<code>tb_*.v</code>	Testbench self-checking que lee los <code>.hex</code> y compara contra el DUT
<code>run.sh</code>	Encadena los pasos: generar → compilar → simular → GTKWave

Flujo de un ejercicio:

1. `gen_vectors.py` emite uno o más archivos `.hex` con los vectores de entrada (`a.hex`, `b.hex`, ...) y los valores esperados (`expected.hex`).
2. El testbench Verilog usa `$readmemh` para cargar esas memorias.
3. Por cada vector, aplica las entradas al DUT y compara la salida contra el valor esperado.
4. Al final imprime PASS o FAIL con la cantidad de casos correctos / fallidos.

Ajustar la cobertura

Por defecto, cada `gen_vectors.py` genera 1000 vectores (bordes + random). Para cambiarlo, exportar `N_VECTORS` antes de correr `run.sh`:

```
# Cobertura rápida (debug)
N_VECTORS=100 ./run.sh

# Cobertura exhaustiva
N_VECTORS=10000 ./run.sh
```

En los ejercicios 5 y 6 la cobertura por defecto es exhaustiva (256 y ~105 casos respectivamente), porque el rango representable cabe entero en memoria. `N_VECTORS` sólo limita el random extra.

Comandos individuales (sin run.sh)

```
# 1. Generar vectores
python3 gen_vectors.py

# 2. Compilar
iverilog -o sim.out tb_*.v *.v

# 3. Simular
vvp sim.out

# 4. Ver waveform
gtkwave *.vcd &
```

Referencias

- Documentación: <https://francof2a.github.io/fxpmath/>
- Código fuente: <https://github.com/francof2a/fxpmath>